CQF Exam Three

Machine Learning in Finance

June 2026 Cohort

October 17, 2026

Abhay Dodiya

Contents

1 Executive Summary **2**

2 Input Data **3**

3 Feature Engineering **6**
3.1 Feature Selection . . . 6
3.1.1 List of Expanded Features: . . . 7
3.1.2 Feature Selection funnel: . . . 7
3.1.3 Mutual Information Math and Example: (Filter Stage) . . . 7
3.1.4 Wrapper and Embedded Method: . . . 9
3.2 Feature Transformation: . . . 9
3.3 Final Selected Feature: . . . 9

4 Model Building **9**
4.1 Various Models . . . 9
4.1.1 XGBoost . . . 9
4.1.2 LightGBM . . . 10
4.1.3 CatBoost . . . 10
4.1.4 Objective function: . . . 10
4.1.5 Regularization Objective: . . . 10
4.1.6 Second Order (Newton) Boosting: . . . 11
4.1.7 L1 and L2 tree scores: . . . 11
4.1.8 Paramter Mapping: . . . 11
4.1.9 Tree Illustration: . . . 12
4.2 Hyper Parameter Tuning . . . 13
4.3 Evaluation . . . 14

5 Conclusion **14**

6 Back testing: **14**

7 Bibliography **15**

1 Executive Summary

This project aims to develop the predictive model to forecast the positive market move (uptrend) of **iShares Semicondutor ETF (SOXX)**. Three gradient boosted decision tree (**XGBoost,LightGBM,CatBoost**) are evaluated.

From ten years of daily data (2015–2025), a 31-column feature library was engineered across trend/momentum, volatility/tail, regime, and macro categories, built on log-returns for stationary (ADF-audited: 27 of 31 stationary). A three-stage selection funnel began with a mutual-information filter that retained 10 features with detectable signal; two model-based stages (RFECV and Lasso) were tested but uninformative, so the 10 features passed directly to boosting. Each framework was tuned on its structural hyperparameter under time-series cross-validation and evaluated out-of-sample via purged walk-forward.

All three models score at or below chance on walk-forward ROC-AUC, with accuracy and precision pinned to the 54% base rate (The base rate defines as the unconditional probability of the positive class, roughly 54% of days were up-days) — a null result consistent with the weak negative daily autocorrelation found in the exploratory analysis.

Finally, The frameworks are statistically indistinguishable, the choice rests on operational and risk grounds: **CatBoost** is preferred for its ordered target statistics and biased-tree regularization, which structurally prevent target leakage and overfitting on weak-signal data.

2 Input Data

The iShares Semiconductor ETF was selected because the sector exhibits a strong uptrend and dynamic volatility. In this chapter, we leverage Yahoo Finance data to analyze and predict upcoming price movements.

The comparison table below highlights the key difference between two of them and mainly it is about how diversified the The iShares Semiconductor ETF is which is good part as it avoids the concentration risk.

iShares Semiconductor ETF (SOXX)	**NASDAQ-100 (QQQ)**
Broad base Semiconductor Supply chain.	Heavy Focus on Mega-cap AI and Hardware Leader.
SOXX caps top holdings at 8%, a rally in NVDA won't lift SOXX as much as it would lift a pure market-cap index, but a crash in NVDA will also hurt SOXX significantly less.	QQQ has higher single-stock tolerance, its performance is heavily dictated by a few massive winners (NVDA, AAPL, MSFT).
Top 10 are 30-35% of index	Top 10 are 70% of index

Table 1: Comparison of two instruments

Tickers	**Company Names**
AMD	Advanced Micro Devices, Inc.
INTC	Intel Corporation
MU	Micron Technology
NVDA	NVIDIA Corporation
AVGO	Broadcom Inc.
MRVL	Marvell Technologies
KLAC	KLA Corporation
ADI	Analog Devices Inc.
AMAT	Applied Material Inc.
TXN	Texas Instruments Ind.

Table 2: Constituents of SOXX as of Oct'26

The daily return histogram (log returns %) for both of them shows the taller peak and fat tail than normal curve. They are **leptokurtic** and not normal, large moves happens quite often Gaussian predicts.

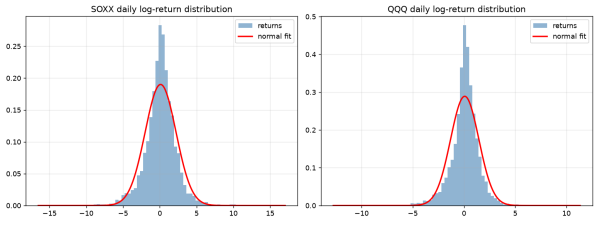


Figure 1: Distribution

The QQ plot exhibit similar pattern where lines bends away at both tails. The effect is more severe in the lower left. This confirms the non-normality; the down tail is heavier than the upside (consistent with negative skew) which means crashes are more likely than the normal model allows.

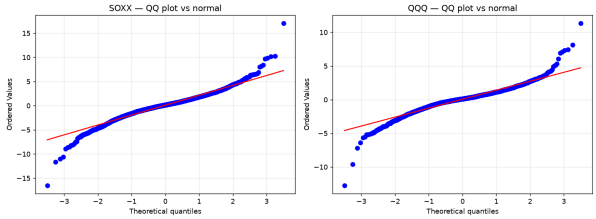


Figure 2: QQ Plot

The 2 X 2 plot below is of Autocorrelation for both index, it focuses on return and squared returns. There are 20 lags and red line is 95% confidence interval band. The returns are roughly **serially uncorrelated** but volatility is highly persistent which is indicative for GARCH type of model.

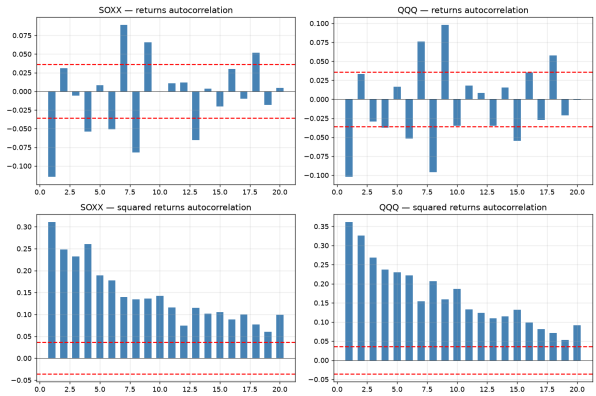


Figure 3: Autocorrelation

If we had invested $1 how it would have behaved over the period of time. SOXX ends materially higher but it also has higher draw down. SOXX delivered higher total return but at higher risk.

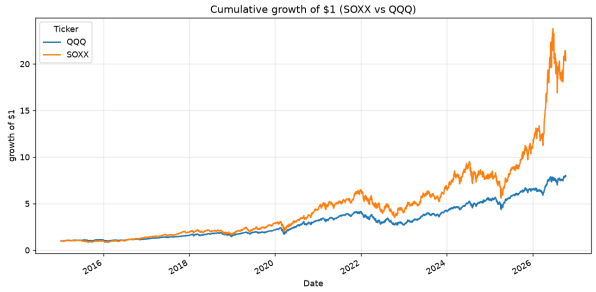


Figure 4: Cumulative Return

If we analyze rolling 63 period volatility annualized over time, it peaks around 2018, 2020(COVID) and 2022 (rate hike) with SOXX persistently higher than QQQ. The risk is time varying and regime-dependent, not constant.

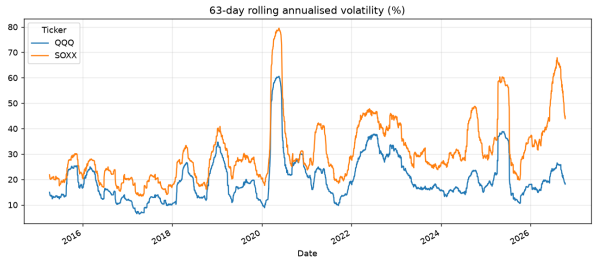


Figure 5: Rolling Volatility

Finally, The exploratory data analysis establish two facts and they are as follows:

- Returns are fat tailed and left skewed.
- volatility is strongly clustered and returns are weakly auto-correlated.

3 Feature Engineering

We are going to go through various features for the modeling purpose. The objective is to first understand them and further assess whether they are relevant or not for our predictive modeling.

3.1 Feature Selection

The features are grouped into various categories and they are as follows a) Trend and Momentum b) Volatility c) Regime d) Macro and Fundamental Indicator. We will first go through their definitions and further assess their usability from our modeling perspective.

- Trend and Momentum:
  - Momentum (multi-horizon) + Δ vol breakout: rolling return over horizon k, $r_t^k = P_t/P_{t-k} - 1$, at several horizons (e.g.10,20,60), combined with a volatility breakout - a surge in volume accompanying the move.
  - Residual Momentum: SOXX return stripped of its QQQ beta, $r_{res,t} = r_{SOXX,t} - \beta r_{QQQ,t}$ over rolling window. This will help us to isolate SOXX-specific drift that is not explained by broad tech benchmark.
  - Savitzky-Golay derivatives: Fit a local polynomial to price path and read off its velocity, acceleration and phase. (Is the trend accelerating, peaking or turning.)
  - Hurst Exponent/Variance Ratio: Hurst H (equivalent variance ratio $VR_k = Var(r^k)/(kVar(r^1)$ measure whether the series persist (H>0.5 trending) or revert (H $<$ 0.5) mean reverting.
- Volatility:
  - Realized Volatility + Δ Vol: Rolling standard deviation of daily return (e.g.21d) plus its changes $\Delta\sigma_t = \sigma_t - \sigma_{t-21}$ this gives us an insight whether risk is expanding or contracting.
  - Left-tail (5%): A flag (or score) built from extreme downside days - the rolling minimum / worst 5% returns on the premise that crashes overshoot and partially revert.
  - Downside Beta: The regression beta of SOXX on QQQ estimated only on down market days (tail dependence).
- Regime:
  - HMM Regime (3 states): a three state Hidden Markov Model fitted to SOXX return, inferring a latent "bull/bear/transition" state and probability of being in each.
  - Kalman local trend + dynamic beta: a Kalman Filter that tracks SOXX local drift (time varying mean) and its dynamic beta to QQQ adopting continuously rather than over a fixed window.
  - Co-integration: The Engle-Granger co-integration residual between SOXX and QQQ log-prices - the extend to which the pairs spread has long-run equilibrium.
- Macro and Fundamental:
  - VIX (implied volatility): The CBOE volatility index and its correlation with SOXX -0.66 i.e.; VIX up and Semiconductors are down.
  - 10Y treasury yield: The rolling correlation between SOXX and 10Y treasury yield varies from -0.48 to +0.64. The regime study of this analysis shows that the rising yield driven by strong growth accompany rising semiconductor, whereas rising yields driven by higher inflation crush semiconductor. This shows a non linear relationship.
  - US Dollar index (DXY): Semiconductor are global cyclical export trade so strong dollar is a headwind.

3.1.1 List of Expanded Features:

The list above further expanded with additional parameters for each of them, for example Momentum (multi-horizon) + $\Delta volbreakout$ is further divided into three sub categories of 10 days, 21 days and 60 days.

Feature	Variation
`mom_10`	lookback $k = 10$ days
`mom_20`	lookback $k = 20$ days
`mom_60`	lookback $k = 60$ days
`dvol`	volume / 20-day mean $- 1$
`resid_mom_20`	β win 63d, cumulative $k = 20$d
`resid_mom_60`	cumulative $k = 60$d
`sg_vel`	SG 1st derivative, win 21, poly 3
`sg_accel`	SG 2nd derivative
`sg_phase`	4-state phase (vel $\times$ accel signs)
`hurst`	R/S exponent, win 126d
`vr`	variance ratio, $k = 20$, win 126d
`rv_10`	ann. rolling std, win 10d
`rv_20`	ann. rolling std, win 20d
`rv_60`	ann. rolling std, win 60d
`drv_20`	$\mathrm{rv}_{20,t} - \mathrm{rv}_{20,t-20}$
`tail5`	rolling 5th pct, win 63d
`tail_shock`	return if below 5th pct, else 0
`down_beta`	β on QQQ<0 days, win 252d
`hmm_bull`	P(bull), 3-state, warmup 504 / refit 252
`hmm_state`	MAP state (0–2)
`kalman_trend`	local-linear-trend slope
`kalman_beta`	random-walk β to QQQ
`spread`	rolling-OLS residual, win 252d
`spread_z`	z-score, win 63d
`coint_beta`	rolling OLS β, win 252d
`vix`	level
`dvix`	daily change
`y10`	level (%)
`dy10`	daily change
`dxy`	level
`rdxy`	daily return

3.1.2 Feature Selection funnel:

The feature selection funnel comprises of three stages and they are as follows:

- Filter: First step is to rank every feature by model-agnostic relevance statistics (the **mutual information** against the target value). The benefit of this method is that it is cheap, uni-variate and ignore the interaction. This is used primarily only to screen out feature with no detectable signal.
- Wrapper: Once we apply the filter the next step would be to evaluate the subset on a baseline model (logistic regression) with cross validation, and keep the combination that optimizes the cross validation objective.
- Embedded: Finally we will use the (Lasso) to shrink irrelevant coefficients.

3.1.3 Mutual Information Math and Example: (Filter Stage)

There are various concepts being used in this section and here is a brief definition of them and why they are being used.

- log_2 : Since our target is a binary outcome (up/down) so bits map directly onto "yes/no" decisions.

- bits: Bits (binary digit) is the amount of information needed to resolve one yes or no question with equal odds.
- Entropy: For a discrete random variable X with probability mass function p_x, Shannon entropy measures the uncertainty (in bits) of X:

$$H(X) = -\sum_x p(x) log_2 p(x)$$

A fair coin has H=1 bit; a deterministic variable has H =0. The conditional entropy measures residual uncertainty in Y once X is known:

$$H(Y|X) = \sum_x p(x) H(Y|X = x) = -\sum_{x.y} p(x, y) log_2 p(y|x)$$

- Mutual Information: The Mutual Information is the reduction in uncertainty about Y obtained by observing X. There are three definitions:

$$I(X;Y) = H(Y) - H(Y|X)$$

$$Y(X;Y) = H(X) + H(Y) - H(X,Y) = \sum_{x,y} p(x,y) \log_2 \frac{p(x,y)}{p(x)p(y)} = KL(p(x,y)||p(x)p(y)$$

- MI is the kullback-leibler divergence between the joint distribution and the product of marginals. It measures how far the variable are being independent.
- Continuous features: The feature we are dealing with are continuous and the target $Y(0,1)$ is binary, so the discrete summation above doesn't apply directly. For a continuous X,

$$I(X;Y) = \sum_{y(0,1)} \int_R p(x,y) log_2 \frac{p(x,y)}{p(x)p(y)} dx$$

The densities are unknown, so the estimate uses the **Kozachenko-Leonenko k-nearest-neighbor estimator** which is part of sklearn $mutual_info_classif$ computes.

- Worked Example: Lets say out problem statement is binary feature X= 'VIX closes below its 20 day mean today.' and the target Y = 'SOXX up tomorrow.', with a joint frequency table (out of 100 days):

	Y = 0 (down)	Y = 1 (up)	total
X = 0 (VIX High)	30	20	50
X = 1 (VIX Low)	10	40	50
total	40	60	100

Table 3: Joint Frequency Table

- Marginal Entropy of Y: With $p(Y = 1) = 0.6$ and $p(Y = 0) = 0.4$:

$$H(Y) = -(0.4 log_2 0.4 + 0.6 log_2 0.6) = 0.9710 bits$$

- Conditional Entropy:

$$H(Y|X = 0) = -(0.6 log_2 0.6 + 0.4 log_2 0.4 = 0.9710$$
$$H(Y|X = 1) = -(0.2 log_2 0.2 + 0.8 log_2 0.8) = 0.7219$$
$$H(Y|X) = 0.5(0.9710) + 0.5(0.7219) = 0.8464 bits$$

- Mutual Information:

$$I(X|Y) = 0.9710 - 0.8464 = 0.1245 bits$$

- **Interpretation:** Given mutual information score of VIX is 0.1245 bits it says that if we simply looking at data without any features we can say that uncertainty of 0.971 bits whether SOXX will go up or not but with additional information of VIX this uncertainty drops to 0.846. This additional or conditional information help us to reduce the uncertainty of predicting next move.

3.1.4 Wrapper and Embedded Method:

- It came down to 10 features after considering the MI Filter stage, further when are passed through remaining two model-based selection stages i.e. Recursive Feature Elimination with cross validation (wrapper) and Lassos Regression (embedded) both failed to identify any meaningful subset. The out-of-fold ROC-AUC remained around 0.48-0.52 across every feature count. This is in line with an expectation given the time series we are dealing with is non-linear in nature. This concludes that neither of them are helpful and hence we retain remaining 10 features as final go for the gradient-boosting model, allowing its non-linear, gain based importance to perform the final selection.

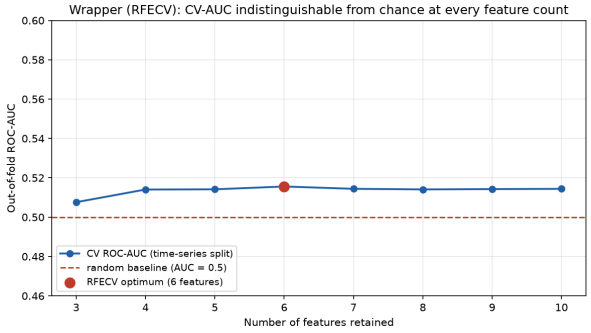


Figure 6: ROC-AUC of features

3.2 Feature Transformation:

- There are two types of transformation which took place so far, first to ensure stationary, every price derived feature was constructed from log-returns $r_t = ln(P_t/P_{t-1})$ which converts a unit-root (I(1)) price process into a stationary return process; The macro level series were likewise paired with their first differences $(\Delta y_{10}; \Delta\text{VIX}; \Delta\text{DXY})$ and augmented Dickey-Fuller test confirmed 27 of 31 features are stationary at 5% confidence interval. In order to process on a common scale for the linear baseline models (logistic regression and Lasso) each was standardized to zero mean and unit variance. The final gradient boosting ensembles requires neither transformation, because decision trees are invariant to strictly monotone transform, so a split on raw level and a split on its log or $z - score$ are identical.

3.3 Final Selected Feature:

vix	VIX Implied Volatility level
dxy	US Dollar index level
dvix	daily change in vix
rv20	annualized realized vol, 20 day window
rdxy	Daily return of USD Index.
y10	10 years US Treasury Yield, % level
vr	Variance Ratio (k=20, 126 day window)
hurst	R/s hurst exponent
kalman beta	time varying beta via Kalman filter
coint beta	rolling OLS beta of log(SOXX) on log(QQQ) 252 days.

Table 4: Final list of features post filtration.

4 Model Building

4.1 Various Models

4.1.1 XGBoost

:

- XGBoost is extreme gradient boosting is build around the concept of ensemble learning. The model starts with a baseline prediction for every features then it calculates the error (residual) between current estimate and the actual values then a shallow decision tree is trained specifically to predict those errors, then new tree's predictions are scaled down by a learning rate and added to the running total to refine the overall model. The algorithm repeat this cycle multiple times and each time focus is to fixing the remaining mistakes left by the collective trees before.
- Example: Imagine you have two variables as your feature to explain tomorrows move for a particular instrument. Let's say we focus on only return and volume change to predict whether instrument will be up or down tomorrow.
- Step 1: The first shallow tree split on the rule and it checks like this $r_{t-1} < 0; \Delta vol_{t-1} < -1\%$ if yes then outcome is down trend.
- Step 2: Check whether this rule make sense; while it might notice some non linear pattern such as $r_{t-1} < -0.1; \Delta vol_{t-1} > 0.1$ the instrument frequently rebound. It builds the rule to correct this specific error.
- Step 3: XGBoost aggregates the scores from all trees, passes the sum through a logistic function, and converts it into a final calibrated probability scores.

4.1.2 LightGBM

- LightGBM grows leaf-wise, it dynamically select the specific leaf node across entire tree that yields the largest decrease in loss, allowing it to converge faster and achieve higher accuracy, though it requires strict controls on the tree depth to avoid overfitting.
- Financial indicators often behave in non-linear fashion. Let's say high momentum combined with low volatility might signal a strong trend, however, high momentum combined with an extreme volatility could signal in exhaustion, here LighGBM's leaf wise growth isolates these specific interaction quickly. The parameter like subsample, introduce stochastic noise reduction, which help prevent overfitting on noisy financial data.

4.1.3 CatBoost

- CatBoost is handling categorical variables natively without manual one hot encoding and it also address the overfitting on noisy data. It automatically converts categories into numerical representation using smart target-statistic encoding on-the-fly avoiding manual processing.

4.1.4 Objective function:

- Gradient Boosted Decision Tree fit an ensemble of K weak learners (trees), each one correcting the residual of previous term.

$$\hat{y_i} = FK(x_{i)}) = \sum_{k=1}^{K} f_k(x_i)$$

For binary classification the score $\hat{y}$ is mapped to a probability by the logistic (sigmoid) function $\sigma(\hat{y}) = \frac{1}{1+e^{-y}}$

4.1.5 Regularization Objective:

- The model is trained by minimizing a regularized objective over all trees.

$$L = \sum_{i=1}^{n} l(y_i, \hat{y_i}) + \sum_{k=1}^{K} \Omega(f_k)$$

- The first term is the loss (binary cross-entropy)

$$l(y, \hat{y}) = -[ylog\sigma(\hat{y}) + (1-y)log(1-\sigma(\hat{y}))]$$

- The second term is per-tree regularization:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 + \alpha \sum_{j=1}^{T} |w_j|$$

where T is the number of leaves, w_j is the score on leaf j and $(\gamma, \lambda, \alpha)$ are the complexity L2 and L1 penalty coefficient.

4.1.6 Second Order (Newton) Boosting:

- The objective is not optimized in closed form, each tree f_t is fitted to the second order Taylor Expansion of the loss around the current prediction.

$$g_i = \frac{\partial \ell(y_i, \hat{y_i})}{\partial \hat{y}_i}, h_i = \frac{\partial^2 \ell(y_i, \hat{y}_i)}{\partial \hat{y_i^2}} = \sigma(\hat{y}_i)(1 - \sigma(\hat{y_i}))$$

- be the gradient and Hessian of the logistic loss. The per-step objective is

$$L^{(t)} = \sum_i [g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2] + \Omega(f_t)$$

4.1.7 L1 and L2 tree scores:

- L2 regularization (λ) applies continuous shrinkage to tree leaf scores by adding a penalty term to the denominator, which scales down every score smoothly toward zero. While it penalizes overly large scores to prevent overfitting. it reduces them proportionally rather than eliminating them, meaning no leaf weight is ever set completely to zero.
- L1 regularization (α) introduces soft-thresholding to promote sparsity by adding an absolute-value penalty that create a kink at zero. Unlike L2, if a leaf's aggregate gradient falls below the threshold, L1 shrinks its score directly to zero, effectively switching off weak or uninformative leaves entirely.
- Complexity (γ) it minimize gain required to add a split, so larger γ yields fewer, shallower trees.

4.1.8 Paramter Mapping:

Penalty	XGBoost	LightGBM	CatBoost
L2 on leaf weights (λ)	reg lambda	lambda L2	L2 leafreg
L1 on leaf weights (α)	reg alpha	lambda L1	-
Minimum Split Gain (γ)	gamma	min gain to split	-

Table 5: Parameter Mapping across three models

4.1.9 Tree Illustration:

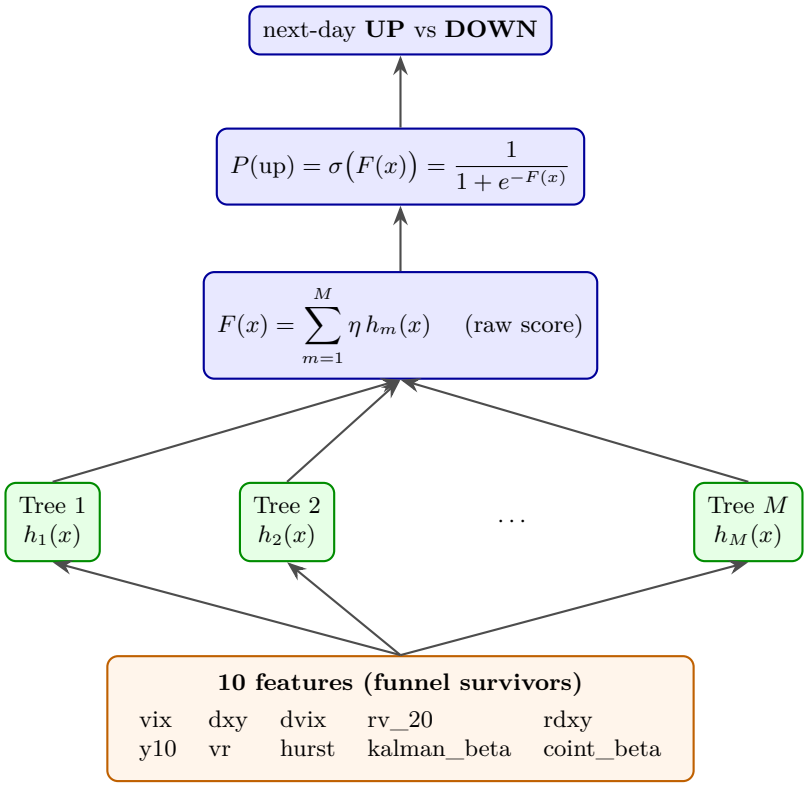


Figure 7: Ensemble flow: the ten features enter, fan out across M trees, the leaf scores are summed, and the result is squashed through a sigmoid into a next-day up/down decision.

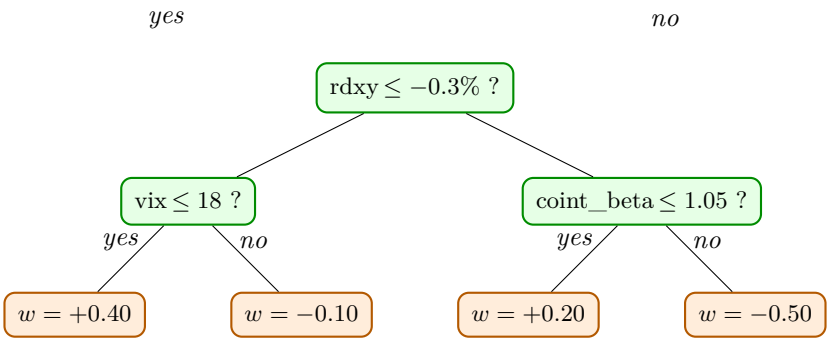


Figure 8: One illustrative tree h_m (a single member of the ensemble). At every node the split feature is chosen greedily by maximum gain over *all* ten features – there is no fixed feature order; different trees choose different features and depths.

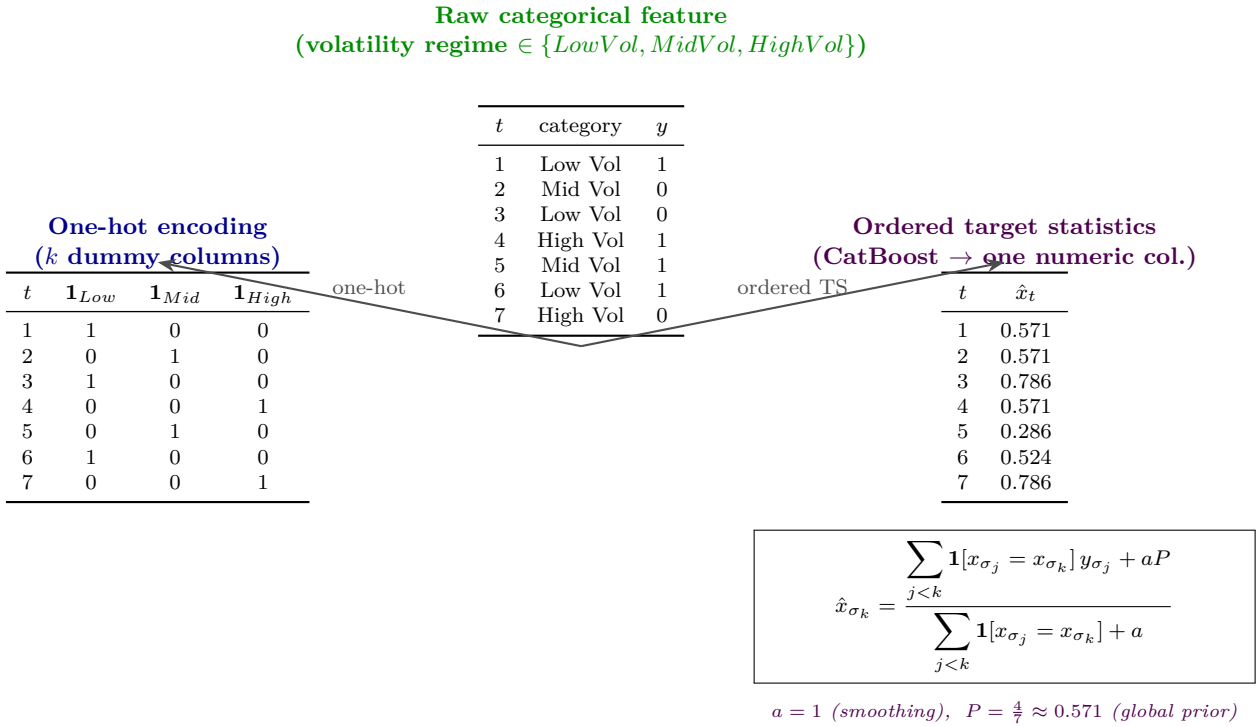


Figure 9: Two ways to feed a categorical feature into gradient boosting. *Left:* one-hot encoding expands the k labels into k binary columns (needed by linear models and naive for trees). *Right:* CatBoost's ordered target statistics replace each label with a smoothed target mean computed only from *prior* rows of a random permutation (prefix $j < k$), so the encoding carries no leakage from the current row. With all-continuous features this machinery sits idle; it is shown for completeness.

4.2 Hyper Parameter Tuning

- The hyperparameter tuning was performed for each model with their structural hyperparameter. It could be tree shape or expanding windows etc. TimeSeriesSplit and ROC-AUC as the objective. The walk forward Cross Validation and exhaustive search would not alter a result that is at a chance. Fundamental parameters are swept for each model; the tuning therefore respects what each algorithm is actually sensitive to.
- Light GBM and XGBoost add tree by maximizing the same second order objective reduction at each candidate split. (the "gain")

$$Gain = \frac{1}{2}[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}] - \gamma$$

- where G,H are the leaf aggregated gradient and Hessian sums and (λ, γ) the L2 and complexity penalties.
- XGBoost grows a full layer per step (breadth first) LightGBM grows the single best leaf per step (best first).
- CatBoost is categorical feature must be encoded numerically before a tree can split on it. The naive encoding - replace each category with the mean of the target for that category. CatBoost fixes this with an ordered statistics: permute the rows randomly then encode each row using the only row before it.

Model	Best Structural Parameter	Time Series CV AUC
XGBoost	max depth = 3, lr = 0.03, n estimators = 200	0.5105
LightGBM	num leaves = 15, lr = 0.03, n estimators = 200	0.5160
CatBoost	depth=4, lr = 0.05, iterations = 200	0.5135

Table 6: Tuning Results

- The CV range is between 0.51 and 0.52 which is marginally above chance.

4.3 Evaluation

- Let $y_i \in \{0, 1\}$ be the true label and $\hat{y}_i \in \{0, 1\}$ the prediction from a threshold on the predicted probability. Define the four cells of the confusion matrix:
- TP $= \sum_i y_i \hat{y}_i$, FN $= \sum_i y_i(1 - \hat{y}_i)$, FP $= \sum_i (1 - y_i)\hat{y}_i$, TN $= \sum_i (1 - y_i)(1 - \hat{y}_i)$.
- Accuracy $= \frac{TP+TN}{TP+TN+FP+FN}$
- Precision $= \frac{TP}{TP+FP}$
- Recall $= \frac{TP}{TP+FN}$
- F1 $= \frac{2 \times Precision \times Recall}{Precision+Recall}$
- FPR $= \frac{FP}{FP+TN}$
- ROC Curve $= TPR_t$ v/s FPR_t
- ROC-AUC $= \int TPRd(FPR)$ Probability a random positive is ranked above a random negative.
- Precision-Recall Curve: Dominant trade off for imbalance execution.
- Average Precision $= \sum_k (R_k - R_{k-1})P_k$ area under the PR curve.
- Evaluation Results (out of sample; walk forward):

Model	ROC-AUC	Avg.Prec.	Accuracy	$Precision_{up}$	$Recall_{up}$	$F1_{up}$
XGBoost	0.4845	0.5356	0.49	0.53	0.46	0.49
LightGBM	0.4909	0.5447	0.49	0.54	0.42	0.47
CatBoost	0.4873	0.5336	0.49	0.54	0.43	0.48

Table 7: Evaluation Results

- All three models score at or below chance (AUC 0.49), with accuracy and 'up' precision pinned to base rate of 54%. The base rate defines as the unconditional probability of the positive class, roughly 54% of days were up-days. The precision-recall reading is the honest one for execution: every models up-signal win only is 53% of the time.

5 Conclusion

- Statistically the outcome of all models are in very narrow range, which makes it difficult to distinguish them based on their predictive power and it should be based on their operational stability and risk.

  Hence, **CatBoost** is the first choice as it structurally eliminate the dominant risk in financial machine learning, also it is the most robust to overfitting without aggressive hand-tuning. its build in leakage protection and regularization makes it the safest choice when signal is weak.

6 Back testing:

- Since our model doesn't have any predictive power, it was worth attempting to try different variations and check if we were able to form any trade on SOXX. We will assess Long only condition along with other parameters for a trade signal. The idea here is to decompose the SOXX into two layer with a clear GBM interpretation: a drift (direction) layer and a diffusion (risk) layer.

- The diffusion filter throughout — enter only when the 20-day realized volatility rv20 is below its trailing (252-day) 80th percentile, which curtails exposure in extreme-volatility regimes.
- The drift layer across four families: raw momentum (10–63d), Savitzky Golay velocity, trend regime gates (Hurst > 0.5 and variance ratio > 1), and residual (SOXX vs QQQ) momentum.
- **Findings:** No drift specification produced statistically significant alpha (all $|t(\alpha)| < 1.6$). The variance-ratio gate kept the strategy in the market only $\sim$5% of the time, confirming that SOXX is mean-reverting rather than persistently trending – consistent with the negative daily autocorrelation observed in the modeling phase. The best configuration is the simplest: 10-day momentum plus the volatility filter. The value is therefore risk reduction — beta-timing, not signal — while the volatility filter curtails exposure in high-volatility regimes as intended.

	Baseline Scenario	SOXX buy-and-hold
CAGR	14.5%	24.5%
Annualized Volatility	17.6%	31.9%
Sharpe Ratio	**0.859**	0.846
Max Draw down	-29.4%	-45.8%
In-Market	46%	100%

Table 8: Back testing summary

7 Bibliography

- Friedman, J. H. (2001). "Greedy function approximation: a gradient boosting machine." Annals of Statistics, 29(5), 1189–1232. (the boosting objective)
- Chen, T. & Guestrin, C. (2016). "XGBoost: A scalable tree boosting system." Proc. ACM SIGKDD, 785–794. (level-wise growth, L1/L2/γ)
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. & Liu, T.-Y. (2017). "LightGBM: A highly efficient gradient boosting decision tree." NeurIPS 30. (leaf-wise growth)
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V. & Gulin, A. (2018). "CatBoost: unbiased boosting with categorical features." NeurIPS 31. (ordered target statistics)
- Shannon, C. E. (1948). "A mathematical theory of communication." Bell System Technical Journal, 27(3), 379–423. (entropy, MI)
- Savitzky, A. & Golay, M. J. E. (1964). "Smoothing and differentiation of data by simplified least squares procedures." Analytical Chemistry, 36(8), 1627–1639.